EDHM_2DMigoto with ED Unofficial Patch

Maintainer review report | 13 November 2026

A candidate replacement for original 3Dmigoto in ED-UP's existing EDHM chain.

Brief and request

EDHM_2DMigoto is an EDHM-focused, XXMI/2Dmigoto-derived fork of 3Dmigoto. The proposal is to use its existing DLL as ED-UP's renamed inner EDHM runtime. The projects remain separate binaries. No code merger or special ED-UP-only build is proposed.

The evidence supports investigating this combination, not endorsing a measured combined improvement. Standalone A/B measurements against original 3Dmigoto 1.4.9 show substantially cheaper changing resource references and fewer SRV creations. Stable references and simple bindings also regress. Removing legacy NVIDIA stereo/profile machinery narrows one area of behavior, while ordinary D3D11 interception remains broad.

Please assess whether the existing ED-UP chain, hook order, state isolation and HUD classification support this exact fork, and reproduce the operation-level comparisons below. No ED-UP-plus-fork execution, FPS measurement, live-game monitoring or combined performance result is claimed by this report.

Compared item	Frozen identity
Candidate	EDHM_2DMigoto 0.1.4-beta.1; DLL numeric version 0.1.4.0
Measured candidate source	`5c74823ff0d21c662e2d659d04afc6624d245509`
Reviewed candidate checkout	`303736e11a416d091c77cdee2d7d6bb95a64a592`; documentation-only changes since measured runtime
Primary original baseline	bo3b/3Dmigoto 1.4.9 beta, `4ce5f2f72b2777223d2e809dcdafec514ac98295`
Historical context	Original 1.3.16; no accepted timing comparison in this audit
ED-UP release	v0.22.0, `ba02b6be7003d3c92aa4ee7f1bfc134aeeab084d`
ED-UP source inspected	`aa765cd9b90d9b3027c6505157c607d151eeaef5`, 18 commits after release

The latest release/head identities were rechecked on the report date. ED-UP's verified game-build context is Odyssey 334925; feature-specific build checks still apply. Source and binary identities are different records: the candidate is a fresh CI Release build, not an assertion about an unspecified installed EDHM DLL.

Sources: candidate repository, original release, ED-UP release, complete earlier measurements.

1. Measured benefits and costs: standalone evidence

ED-UP was absent from every measurement in this section. These are candidate mechanisms worth testing beneath ED-UP, not combined-stack speedups. Negative change means lower cost; brackets contain descriptive 95% bootstrap intervals.

Workload and backend	Original 1.4.9	Candidate	Change [95% interval]
Changing reference, common production component, WARP	1,406.40 ns	161.04 ns	-88.5% [-88.9, -82.1]
Stable reference, same component	73.91 ns	89.11 ns	+20.6% [+18.7, +85.5]
Changing HUD-like reference + draw, actual DLL, Windows WARP	12.789 us	9.122 us	-28.7% [-40.4, -25.1]
Stable HUD-like reference + draw, Windows WARP	6.829 us	7.476 us	+9.5% [+6.7, +12.0]
Changing HUD-like reference + draw, actual DLL, AMD/Proton	411.96 ns	288.35 ns	-30.0% [-33.5, -22.9]
Stable HUD-like reference + draw, AMD/Proton	101.51 ns	210.86 ns	+107.7% [+96.4, +132.8]
Passive stable PS binding, Windows WARP	40.31 ns	61.39 ns	+52.3% [+49.6, +68.3]
Passive stable PS binding, AMD/Proton	4.70 ns	16.06 ns	+241.8% [+194.7, +266.9]

Mechanism and memory evidence

In the common component fixture, changing references reduced Create*View calls from **10,000 to zero** per sample. Clean native lower-layer traces of the actual-DLL HUD fixture recorded **98 versus 33 SRV creations**, including the same 33 fixture SRVs: **65 extra creations versus zero**. Both DLLs made the same 162 buffers, four 2D textures, two 3D textures, 129 draws, one dispatch and 68 CopyResource/Map/Unmap calls in the corresponding HUD capture.

These are lifetime call counts, not simultaneously live objects. The fixture has 3,072 logical bytes of primary resources and 1,024 bytes of validation staging. Fewer views did not mean fewer backing textures or measured lower VRAM. Additional metadata queries were observed in the candidate and are a plausible investigation target for its stable-path cost.

The DLL file is 2.90% smaller: 3,241,472 versus 3,338,240 bytes. Whole-process private commit after the passive WARP workload was 7,485,440 versus 7,837,696 bytes (-4.5%, interval [-8.0, -0.8]); this includes the caller and software backend. The HUD memory interval crossed zero. Wine PrivateUsage returned zero and was classified unavailable. Neither disk size nor private commit is VRAM.

Method and practical limits

Four processes per variant/profile ran in A-B-B-A-B-A-A-B order. Summaries are medians of process medians; repeated timings discard sample zero. Components used ten retained samples; DLL probes used six. Exact independent n=4 bootstrap median ratios produce the intervals; they are one-host repeatability estimates, not cross-hardware guarantees. Windows used software WARP; Proton used physical AMD RADV STRIX_HALO, Mesa 26.0.8 and Proton 11.0-2e/DXVK 2.7.1-498.

All 36 accepted DLL timing processes, five native traces and the component result checks passed. Six controlled caller faults demonstrated checker sensitivity. No per-commit causal attribution or ED-UP net saving follows.

Evidence: raw samples, hashes and reproduction, complete numerical summary.

2. Surface area: what changes and what remains shared

Boundary	Difference from original 3Dmigoto	Relevance beneath ED-UP
Legacy NVAPI stereo/profile controls	Linked startup/profile and stereo-parameter path removed	Less legacy vendor-specific behavior to compose; actual benefit depends on original's configuration.
Normal device/context and draw interception	Retained	World Draw/DrawIndexed still pass BeforeDraw/AfterDraw, even without an EDHM override.
HUD shaders, parameters and resource references	EDHM behavior retained, with adapted XXMI and local fixes	Direct overlap with ED-UP's UI classification, jitter/depth and temporal history.
Resource/view handling	Compatible view reuse and lifetime/range safeguards	Lower creation work in measured patterns; same resource identity and descriptor remain essential.
Loader, DXGI factories and swapchain	Hooks/wrapping retained	Initialization, module ownership, Present and state restoration need combined validation.
Wine/Proton setup	Explicit Wine-aware chaining/defaults	More explicit backend selection; original can also work when configured correctly.

Removing NVIDIA 3D Vision behavior does not remove ordinary 3D rendering or make the DLL a HUD-only interception layer. ED-UP's OpenXR/game stereo path is different from legacy driver stereo. Upscaling also operates in flat mode. The claimed benefit is narrower legacy behavior, not proven fewer collisions. If original's stereo behavior was already disabled, the practical difference may be smaller. Original's D3D11 core is not NVIDIA-only: both measured DLLs passed the named AMD/Proton fixture.

Important: a renamed inner DLL also installs entry detours

Our same-directory target guard accepts `d3d11_edhm.dll`. During DLL attach, however, the module differs from `GetModuleHandleA("d3d11.dll")`, so inherited `HookD3D11` detours D3D11CreateDevice and D3D11CreateDeviceAndSwapChain in the module named `d3d11.dll`, normally ED-UP. Original 1.4.9 contains the same renamed-load branch. This is not a newly discovered candidate-only defect.

`load_library_redirect=0` **does not disable these entry detours, the installed LoadLibraryExW hook or DXGI factory hooks.** It disables configured redirection. ED-UP defers its chain load and contains reentrant-create fallback and outermost attachment guards. These are relevant protections; actual initializer/call routing still needs checking. No detour-related failure has been demonstrated.

The intended ownership diagram is `Elite -> ED-UP -> inner EDHM -> backend`. It is not a complete hook graph, nor proof that the two modules touch disjoint objects. Do not remove draw/state interception merely to shrink that diagram: EDHM tracking can depend on calls outside the final HUD draw.

Sources: candidate loader and detours, original renamed-load branch, candidate draws, ED-UP chain and create guards, legacy comparison sources.

3. Proposed configuration

Upstream documents the outer/inner rename and `advanced.real_dll` setting. The additional d3dx.ini settings below are source-informed recommendations for this review; they are not presented as maintainer-certified upstream requirements. Merge them into the existing files, retaining all EDHM includes and settings.

```
Odyssey product directory, beside EliteDangerous64.exe:
  d3d11.dll              ED-UP's original filename
  d3d11_edhm.dll         candidate, renamed from d3d11.dll
  edvr.ini              shared ED-UP settings and chain
  edvr-flat.ini         flat settings, if present
  d3dx.ini              EDHM configuration, retained
  EDHM-ini/             same EDHM configuration/assets
  ShaderFixes/          same EDHM shader pack
  d3dcompiler_47.dll     required compiler redistributable
```

In shared **edvr.ini**:

```
[advanced]
real_dll = d3d11_edhm.dll
```

An existing nonempty `real_dll` in edvr-flat.ini takes precedence for flat launches; `none` disables the chain. Remove an unintended override or deliberately make it agree. A stale `[chain] d3d11=...` troubleshooting example is not the implemented setting. Keep ED-UP's current VR runtime layout when testing VR; renaming the EDHM DLL does not replace ED-UP's OpenXR setup.

In **d3dx.ini**, merge:

```
[System]
load_library_redirect = 0
allow_platform_update = 1

[Device]
upscaling = 0
```

Setting	Reason
load_library_redirect=0	Avoid later system D3D11 loads being substituted with the game-local outer ED-UP module. Does not turn off installed hooks.
allow_platform_update=1	Permit Device1/Context1 access used by ED-UP's flat context-state isolation. Already enabled in our template.
upscaling=0	Keep our legacy backbuffer/swapchain upscaler inactive while ED-UP supplies reconstruction. Already our default.
proxy_d3d11 unset	Avoid a reverse link from the inner DLL back to ED-UP.

Retain EDHM's `ini_params=120` and `stereo_params=-1` under Rendering where those are the shader pack's expected settings. For the original A/B baseline, keep its legacy stereo/profile behavior disabled consistently. Record any baseline-specific settings rather than silently changing them between trials.

On Proton, load the outer game-local native d3d11 through the game's existing Proton setup and retain DXVK in the prefix. Do not place DXVK's D3D11/DXGI binaries into this product directory to establish the chain. Record all DLL sidecars and overrides; extra mod layers belong in separate comparison cases.

Sources: supported EDHM installation, flat/shared setting precedence, candidate defaults, Wine setup.

4. AA, vendor and compatibility boundaries

ED-UP engine	Implementation	Candidate review boundary
TAA	ED-UP's temporal history, depth and motion	Shared state, projection/UI classification and history behavior.
DLSS / DLAA	NGX plus nvngx_dlss.dll on eligible RTX hardware	Removing legacy 3D Vision does not remove NGX; correct inputs/runtime are still required.
FSR 3.1	Community FidelityFX-SDK-DX11 upscaler compiled into ED-UP	Official newer SDK API restrictions do not rule out this D3D11 port.
XeSS-SR	Intel D3D12 SDK over shared D3D11/D3D12 resources	Device5, Context4, shared textures/fences, adapter match and bridge copies/waits.

ED-UP's temporal engines are not themselves frame generation. Native-size DLSS is the DLAA quality case; it adds processing without reducing scene pixels. A lower render resolution can save GPU shading time, with reconstruction, UI, motion and synchronization consuming some of that saving. CPU-bound submission is a different limit. Higher UI-quality targets and HUD AA can add substantial surface storage and work.

Our standalone AMD/Proton result does not certify this combined stack, NGX or XeSS's bridge. Native NVIDIA/AMD/Intel Windows execution, Linux NVIDIA/Intel, VR and SDK interoperability are unexecuted cells for this investigation. The compiled D3D11 wrapper and a backend's feature availability are separate questions. Original 1.3.16's failed Proton preflight is context only, not a universal compatibility diagnosis.

Existing protections and known reports

- ED-UP has measured EDHM projection/UI variants. Unknown replacement variants can still miss jitter cancellation or UI-depth treatment. Changes in theme, shader pack or reload should be distinguished from changes in the DLL binary.
- The historical HUD-depth problem in issue #90 was marked fixed in 0.20.0 RC2. It is not proof that current EDHM always ghosts or swims.
- PR #111, merged before v0.22.0, restricts optional SwapChain3 probing to avoid bypassing a third-party Present wrapper. Our swapchain also exposes raw higher interfaces, so keeping that guard matters.
- ReShade's loader guard is a separate historical interaction. ED-UP contains system-module preloading and IAT mitigations. Open issue #118 still contains theme reports without a single established cause for every configuration.
- A post-v0.22.0 flat held-tool camera/AA repair exists on main. Record the chosen ED-UP revision; unrelated upstream stand-downs must not be attributed automatically to this fork.

Do the initial comparison without ReShade or extra OpenXR layers; add the actual user layers one at a time afterward. This isolates responsibility and is not a statement that those layers are inherently incompatible.

Sources: ED-UP temporal settings, XeSS bridge, HUD-depth issue #90, swapchain repair #111, theme reports #118, version-specific source review.

5. Independent reproduction: controls and comparison

Use isolated fixtures/copies, fixed EDHM assets and one frozen ED-UP build. First reproduce the published standalone evidence; then compare the two inner DLLs under the same ED-UP configuration. These are proposed maintainer checks, not test programs newly written or executed for this report.

Case	Runtime arrangement	Purpose
A	Original 1.4.9 alone	Reproduce original operation costs and correctness.
B	Candidate alone	Reproduce candidate costs and correctness.
C	ED-UP + original 1.4.9	Existing-style chain baseline.
D	Same ED-UP + candidate	Direct comparison for the proposed pairing.
E, optional	ED-UP alone	Attribute ED-UP-only behavior; not an EDHM color-equivalence baseline.

The decisive combined comparison is **D versus C**. Results B versus A are context, not a substitute. C/D/E have not been run by this investigation.

Keep the comparison interpretable

1. Keep the EDHM shader pack, theme, compiler, ED-UP revision, engine, render/ output sizes, diagnostics and backend identical in C and D. Record hashes. Use only configuration directives supported by both inner DLLs for the common comparison; test fork-only commands separately.
2. Place either inner DLL under the same d3d11_edhm.dll filename, beside the caller. Preserve the previous file and restart between variants. Confirm the actual module path and chain selection rather than inferring it from names.
3. Declare sidecar policy. Original ships an NVAPI proxy; the candidate has no linked requirement for it. Record its presence/hash. Changing that policy is an additional experimental variable, especially for vendor SDK loading.
4. Reuse the published A-B-B-A-B-A-A-B process ordering, warmup rules and per-process summaries for operation timings. Do not pool different GPUs, runners or backends. A later rebuilt binary gets its own receipt/hash.
5. Capture API traffic separately from timing. Lower-layer tracing changes execution and mapped-memory handling. Record explicit view/resource factories, copies, readbacks and logical bytes; avoid interpreting them as VRAM.
6. Establish feature engagement. A generic standalone caller may cause ED-UP's game-specific producers to stand down. A clean exit or fallback output is only a plumbing result, not proof that DLSS/FSR/XeSS or stereo actually ran.

Reproduce the numerical summary without D3D11

Download the pinned archive and the standard-library summarizer from benchmark commit `c66db49af0602286ee448557ca2123e5a35d120c`:

```
python Benchmarks/summarize_original_comparison.py \
  /path/to/original-comparison-2026-11-13 \
  --output /tmp/original-comparison-summary.json
```

Check against archived summary.json and SHA256SUMS. The linked guide includes native build/run steps. These operation-level claims require no FPS or live-game monitoring evidence.

Sources: archive guide and raw files, pinned summarizer.

6. Validation outcomes for maintainer review

Area	Observable outcome to protect
Loading and entry detours	Both create-device exports reach the intended backend; initialization terminates; ED-UP attaches to the intended device once; no silent chain bypass. Record module ownership/call order.
Interfaces and Present	Required Device1/Context1 queries succeed; optional higher swapchain queries do not discard EDHM's Present path. Theme/reload processing remains active.
Resources and views	References bind the intended backing resource, format, stride/range and view kind. Stable/changing sources and failure paths preserve their documented behavior.
Context state and maps	IniParams t120 and unrelated bindings survive ED-UP isolation/restoration. WRITE_DISCARD data reaches the lower layer before Unmap, including cached/bounced mappings.
Rendering and classification	Known offscreen color/depth/compute/sentinel outputs remain correct where unchanged. Actual EDHM variants receive the intended HUD treatment; missing classification is reported rather than counted as success.
Engines, reload and shutdown	Selected engine engages instead of falling back; resizing, history resets, theme/shader reload and shutdown retain valid ownership. XeSS sharing/fences complete without stale output.

A temporal filter intentionally changes pixels and history. Do not require byte-identical filtered output to AA-off; use a controlled expected image or engine-specific reference/tolerance. Exact comparisons remain appropriate for untouched resources, selected descriptors and known sentinel values.

A generic offscreen fixture cannot certify every game-specific producer or VR path. Flat checks do not validate stereo. Functional viewing by the maintainer may supplement these checks, but no FPS/live-monitoring evidence is needed for the narrow performance claims, and none has been supplied here.

Questions for the ED-UP maintainer

1. Does the renamed inner DLL's inherited entry detouring preserve ED-UP's deferred chain initialization, reentrant-create handling and outermost-only attachment, including later device creation?
2. Are the current shader recipes and UI guards sufficient for the exact EDHM assets used here, including t120 handling, theme changes and shader reload?
3. Does state isolation restore both GPU state and each wrapper's state shadows when newer interfaces expose raw objects?
4. Is redirection zero the preferred configuration for this fork? Would a narrowly scoped opt-in hook control be useful only if a concrete case shows a need for it?
5. Can C/D comparisons confirm any end-to-end reduction in operation cost without losing feature engagement, output correctness or ownership safety?

Decision rule

Keep one maintained candidate DLL and document the ED-UP configuration. There is no demonstrated production defect requiring a special variant. If a combined failure is reproduced, fix the specific boundary or add a small opt-in mode with a focused check. A separate binary is warranted only by an established incompatible implementation requirement, not an assumed benefit from fewer hooks. The historical original and candidate share several hook behaviors.

Sources: our runtime contracts and tests, swapchain boundary, wrapper hook policy.

7. Binary provenance and handoff

Measured x64 DLL identities

Candidate SHA-256, 3,241,472 bytes:

`bcad31bec60bd072ebcf5f1d152593ad8cf359d4ddfafbbb38fcb3a23611f7e9`

Original 1.4.9 SHA-256, 3,338,240 bytes:

`791aa68b9f8c0742c040f9ed5fa61aa0b4d3f9ee7b9e1865bf82ed6b2254378f`

Original release archive SHA-256:

`d5d05e78f0f6df7cce8ef10c669c088db6b8dbe1428ce783d8c3c93ac842dce1`

The measured candidate came from CI run 43441909497, artifact `d3d11-release-x64-5c74823ff0d21c662e2d659d04afc6624d245509`. CI artifacts expire. If unavailable, build the pinned source with VS 2022/MSVC v143 and a Windows SDK, Release x64, following the repository's build guide. Record the compiler/SDK and resulting hash; rebuilding later does not guarantee byte identity. Build the DirectX11 project or solution, not an unrelated full XXMI tree. Keep the EDHM shader/configuration package separate and record its identity. This report does not redistribute an EDHM asset pack or ED-UP binary.

Useful handoff links:

- Pinned candidate source and build guide.
- Full provenance, sidecars, hosts and excluded attempts.
- Benchmark source and standalone build/run workflows.
- Detailed ED-UP source assessment.
- Upstream issue/release research and version distinctions.

Short brief suitable for forwarding

We maintain an EDHM-focused XXMI-derived 3Dmigoto build and would like your assessment of using it as ED-UP's existing renamed EDHM chain target. Against original 1.4.9, standalone tests show much lower changing-reference/view-creation costs, with stable-binding regressions also documented. Legacy NVAPI stereo/ profile machinery is removed, but normal D3D11 interception and renamed-load entry detours remain. We propose your documented real_dll chain with our load redirection disabled, modern interfaces enabled and legacy upscaling off. Please review hook order, state restoration and the actual EDHM shader variants, and compare both inner DLLs beneath one fixed ED-UP build. We have not measured this combined stack or claimed an FPS/VRAM gain. The report links source pins, raw samples, hashes, configurations and reproducible checks for independent assessment.

Status: report and proposed validation procedure only. No message was posted to the upstream project, and no combined compatibility endorsement is implied.